

When Hostnames Become Code: Detecting Persisted Hostname Injection at Production Scale

Abstract—Hostnames often outlive the parser that rejected them. Values from HTTP Host/SNI headers, DNS lookups, and login telemetry can be blocked at ingress, copied into observability systems, and later consumed by shells, SQL generators, templates, alert actions, or policy logic. We study this deferred cross-parser reuse and present Contrastive Cone Divergence (CCD), a production detector for hostname-borne injection that trains without pooling raw tenant telemetry.

CCD models behavior-preserving rewrites instead of individual spellings. It embeds each recorded host value, scores its cone neighborhood with a split-conformal fixed-FPR likelihood-ratio sketch, and can attach finite-edit decision-stability checks scoped to the deployed normalizer, score path, threshold, and edit manifest. On a 200.3M-row, 835-tenant production replay, CCD reaches 93.5% TPR at 10^{-4} FPR while staying within a 0.60 ms full-path latency budget. In a three-month head-to-head against maintained regex and WAF controls, it surfaces verified hostnames with executable semantics observed downstream, including 850 missed by deployed controls under detector-hidden review. We release the Hostname Injection Benchmark (HIB), a tenant-disjoint, time-forward replay with conservative benign, positive, and unresolved labels.

1. Introduction

An edge proxy can reject a request and still preserve the dangerous part of it. Hostnames label endpoints, steer routing, and annotate telemetry, so they are usually treated as inert metadata once routing has failed. In modern observability pipelines, however, the rejected string may survive in logs, warehouses, alerts, and dashboards, where it is later passed to shells, templates, policy engines, or queries.

```
snowflake.connector.errors.ProgrammingError:
  002141
  (42601): SQL compilation error
    Unknown user-defined function
      DBMS_PIPE.RECEIVE_MESSAGE.
Traceback (most recent call last):
  File "example.py", line 134, in <module>
    example.com' | dbms_pipe.receive_message(
      chr(98) || chr(98) || chr(98), 15) ||
      '.online.cmgroep.local
```

Figure 1. Hostname injection can propagate long before confirmed impact. The payload above is line-wrapped for readability and was a benign delay probe. Routing rejected it, but telemetry retained it, and a later Snowflake write/query parsed the retained bytes as SQL syntax before failing with the compilation error shown. The event records deferred cross-parser reuse, not compromise.

The security boundary then moves from the router to the consumer that reads the stored value.

The security question shifts at that point. An attacker-controlled value that enters through an HTTP Host/SNI field, DNS-derived field, or DHCP hostname can become dangerous long after the request path has finished, because a downstream consumer may parse the retained bytes under a richer language than the edge ever allowed.

Figure 1 shows the query case that motivated the study. The captured host string placed a benign `DBMS_PIPE.RECEIVE_MESSAGE` delay expression between an `example.com` prefix and an `online.cmgroep.local` suffix, using SQL concatenation delimiters around the expression. It was invalid as a route, but it survived in telemetry and entered a warehouse write/query path, where the database parsed the retained bytes as SQL syntax and returned a compilation error. The issue was fixed after investigation. HTTP 400 stopped routing; it did not stop the retained value from reaching a query parser. In diagnostic and alert-automation cases, retained host fields likewise became input to command runners, templates, SQL builders, URL lookups, alert actions, or policy rules.

We call this pattern **deferred cross-parser reuse**: a field is validated or rejected under one grammar, persisted as operational data, and later interpreted under a different grammar with executable semantics. The failure is temporal as well as syntactic: the component that records the bytes and the component that executes their meaning do not share a parser boundary.

Prior work and public incidents show why host-like fields are fragile trust boundaries. They have created parser ambiguity in HTTP, authentication, redirects, DNS rebinding, and DNS tunneling [1], [2], [3], [4], [5], [6], [7], [8]. Unsafe hostname expansion has also produced command-template injection in deployed systems [9], [10], [11], [12], [13], [14], [15], and recent advisories describe Cockpit remote-login command execution, enterprise git push metadata-to-RCE, and Koa `ctx.hostname` userinfo parsing [16], [17], [18]. We do not train on these cases. They instead anchor the same failure modes we see in private telemetry: encoded shell expansion, delimiter-injected metadata fields, and disagreement among authority parsers about where the userinfo and hostname end.

Hard negatives are abundant because invalid DNS syntax is ordinary telemetry. Operational hostname fields contain malformed values from parser failures, scanners, redirects, copy/paste errors, tracking systems, Unicode/IDNA edge

cases, URL fragments, and partially decoded telemetry. In our replay, 1,047,309 rows contain common shell metacharacters; among DNS rows alone, more than 672K such rows are benign or unresolved. A detector that fires on \$, ;, braces, quotes, or percent encodings alone would drown in background noise. The distinction has to be behavioral: whether the string still carries an action a later interpreter would perform, and whether it reached a place where that action could matter.

The mismatch is between the language that recorded the value and the language that later consumes it. A string can be invalid for routing, or harmless there, while remaining meaningful to a shell, SQL engine, template engine, or policy language after interpolation. Accepted-hostname validation is too early and too narrow for this later reuse. The useful intervention point is the retained field, before helper code interpolates it. In this setting, %24%28id%29, delimiter rewrites, and prefix-inserted SQL fragments can preserve the same payload while benign UUID-like labels, URL fragments, and parser residues vary for unrelated reasons.

Our approach is to model payload behavior rather than enumerate spellings. Rewrites that preserve a shell, SQL, template, or lookup action should stay close; malformed benign residues that merely share punctuation should not. The decision object is therefore a behavior-preserving rewrite class, not an individual string.

We call the resulting hostname detector **Contrastive Cone Divergence (CCD)**. At its core is a likelihood-ratio sketch: a normalized string is embedded, summarized by the cones it activates, and compared with benign and executable-reference distributions. Optional certificates are deliberately narrower than safety claims; they state only that the emitted CCD decision is stable over a finite edit ball under the fixed deployed configuration: normalizer, encoder and cone partition, score path, threshold, and edit-manifest version.

These deployment constraints shape the detector. CCD cannot collect raw tenant payloads for training or spend analyst time on speculative tails. It trains instead on taxonomy-guided synthetic payloads plus aggregated benign statistics, refreshes only the benign reference and threshold as names drift, and matches real-payload training within 2.4 points at a 10^{-4} false-positive rate. We then test the fixed detector under predeclared alert budgets, family holdout, drift, mimicry, replay, and live overlap with deployed controls.

At production scale, the abstraction remains practical. CCD outperforms the replay LLM baselines at the fixed false-positive rate while fitting the latency budget, and it surfaces obfuscated positives beyond maintained signatures. The verified positives include command callbacks, code and template execution, SQL delay or failure behavior, and execution failures across CWE-78/89/94/1336-style consumer classes.

The scope is delayed reuse of host fields, not protocol-layer exploits (see §3).

Contributions. This work makes five contributions:

(1) **We identify deferred cross-parser reuse.** Edge-rejected and malformed host strings can survive in

telemetry and later become shell, template, query, or policy-language input.

- (2) **We release a production benchmark.** The Hostname Injection Benchmark (HIB) is a 200.3M-row, 835-tenant replay with time-forward splits and labels for host-like values that reach downstream tooling.
- (3) **We design CCD.** The detector scores behavior-preserving rewrite classes with a likelihood-ratio sketch, calibrates decisions to a deployment budget, and attaches finite-edit decision-stability certificates only within the fixed deployed configuration.
- (4) **We evaluate under operational constraints.** Replay, calibration, latency, family holdout, drift, hostile mimicry, and live overlap test the same detector under predeclared alert budgets.
- (5) **We validate scope and reproducibility.** Source-code reachability checks across a 50-repo public corpus, public-real replay, downstream evidence, and de-identified artifacts show what the detector covers and what remains outside the claim.

We follow the host value after routing has rejected it: §3 defines the artifact, §4 learns rewrites that preserve executable meaning, §8 evaluates CCD under replay, robustness, and live-overlap constraints, and Appendix E states the reproducibility boundary.

2. Related Work

2.1. Network, Web, and Domain Abuse

DNS and domain-abuse work covers dangling records, DNS tunneling, reputation attacks, malformed resolver inputs, per-query sanitization, and malicious-domain detection [7], [8], [19], [20], [21], [22], [23]. Web-input work finds parser and sink mismatches in HTTP implementations, WAFs, OAuth redirects, and SSRF-capable URL consumers [1], [4], [24], [25]. Those settings reason about a name while it is still a network object or live request value. Our setting is the operational afterlife of that value: it is no longer used for routing, but it may still carry command, query, template, or URL-authority syntax.

The closest setting is HALO, which studies host-name poisoning in web applications: a client-controlled host value is trusted by server, framework, or application logic and then flows to sinks such as redirects, OAuth links, SSRF, or authentication decisions [26]. HALO governs authority *within* a request. We study the next stage in the lifecycle, when warehouse jobs, diagnostic helpers, and alert actions handle the stored field as ordinary operational data. This is also related to stored or second-order injection, but the storage layer here is cross-service telemetry and the consumer is often operational tooling outside the original application. That shift changes both placement and evidence: malformed authority syntax is not enough, and a verified positive must reappear in a later record and exercise the corresponding harness.

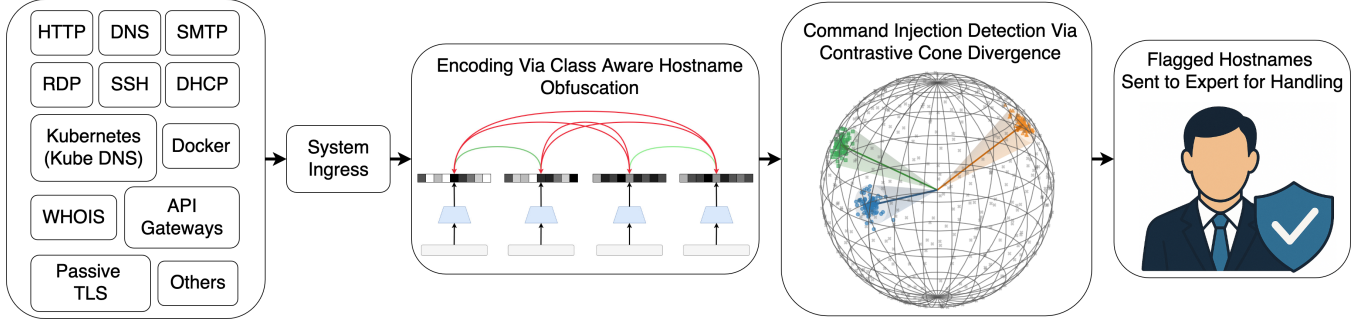


Figure 2. **CCD system architecture.** Training learns host embeddings and benign/payload references from synthetic payloads and aggregate benign statistics. Inference canonicalizes a logged host, embeds it, scores its cone neighborhood, and alerts above the tenant threshold; large-margin positives can receive finite-edit decision-stability certificates

2.2. String, Log, and Instruction Detectors

Surface-form and post-hoc detectors target adjacent stages. *Surface-form* detectors (URLNet, urlBERT, CSI [27], [28], [29]) classify locator strings from their observed spelling. In retained host telemetry, broadening such detectors to catch $\{IFS\}$, double decoding, backticks, or quote/comment breakers also admits large volumes of malformed-but-benign traffic with the same characters.

Post-hoc provenance and log-anomaly systems (FLASH, KAIROS, R-CAID, DeepLog, LogBERT [30], [31], [32], [33], [34]) reconstruct activity after it has occurred. CCD sits earlier in the telemetry path: it decides on one high-risk metadata field before that field drives automation.

Prompt-injection work provides a useful analogy: untrusted text can become instructions for LLM agents and web plugins [35], [36], [37]. Our interpreters are older and less forgiving. Shells, SQL engines, templates, and policy logic react to byte-level hostname obfuscation, while ordinary host artifacts supply millions of hard negatives with the same punctuation.

2.3. Methodological Foundations

Existing robustness and representation tools provide useful baselines, but retained telemetry strings require a different contract: the statistic used for scoring must also be the object bound to calibration and finite-edit decision-stability checks.

Adversarial Robustness. Randomized smoothing [38] typically certifies continuous domains (images) or text via random token substitution. Hostname artifacts need a different unit: edits such as percent decoding, label splitting, prefix insertion, and quote/comment fragments. CCD uses edit-channel search only to prioritize likely high-impact obfuscations; emitted decision-stability certificates are accepted by deterministic margin or enumeration checks over the deployed score threshold. The certificate is tied to the normalizer, cone sketch, threshold, and edit-manifest version that produced the decision. In our evaluation, this finite-edit decision-stability check applies to 98% of detected positives at $K=6$.

Contrastive Learning. Widely used in security for intrusion detection [39], [40], standard contrastive approaches rely on class-agnostic augmentations. Our representation uses class-aware augmentations to enforce semantic invariance for injection payloads while preserving benign diversity. In ablations, this asymmetry separates heavy attacker obfuscation from natural benign variation and improves recall over standard contrastive learning by 2 percentage points (§8.3).

3. Threat Model and Problem Formulation

The object we classify is the recorded host value, not the request that produced it. By the time CCD runs, routing and request-time parser decisions have already happened. The string has been saved, but it has not yet been handed to the operational toolchain that may treat it as a command, query, template, or lookup input.

3.1. System Model: Recorded Host Values

Ingress controllers (e.g., Nginx) often reject RFC-non-compliant requests (e.g., those containing `|` or `$`) from routing, while logging and telemetry systems still record the value. Storage and analytics pipelines such as Snowflake or Splunk can later pass it to automation (Figure 1). Our case-study replay shows the same pattern in shell-based diagnostics during incident triage, alert actions that enrich or “lookup” host fields, and templated SQL in dashboards and scheduled reports.

Recorded host values. In telemetry, the “hostname” field is often a raw host artifact rather than a strict DNS hostname. It may include URL-like prefixes (e.g., `https://`), partial decoding/encoding (e.g., `%HH`), layered encodings, confusables, or other preprocessing debris. We treat that raw string as untrusted user input regardless of upstream routing status.

This population is broader than DNS validity. In benign traffic we observe malformed values produced by instrumentation bugs, broken clients, scanners, tracking identifiers, copy/pasted URLs, encoding residues, and tenant-specific naming conventions. Such values can be nearly arbitrary strings and often overlap superficially with injection syntax. Consequently, neither the detector nor the labeling protocol treats punctuation or marker-like substrings as proof

of executable behavior; the security-relevant distinction is whether the string preserves an action that a downstream parser would execute and whether it reached that parser.

We write this raw artifact as $r \in \mathcal{R}$. CCD operates directly on r and outputs a binary decision $h(r) \in \{0, 1\}$ indicating whether the value carries executable structure for downstream reuse. All threat-model definitions and evaluations use r as recorded, rather than assuming any particular upstream sanitization.

3.2. Labels and Scope

A *verified positive* has executable behavior, reappears in a downstream record, and exercises the corresponding harness. The production-replay analyses use 363,401 verified executable-semantics positives; the live deployment reports operational overlap and tenant-facing response separately from replay accuracy.

3.3. Adversary and Scope

Adversary model. The adversary crafts host strings with printable characters and obfuscations such as homoglyphs, encodings, delimiter edits, and prefix manipulation. The string need not be a valid DNS name. Its goal is to survive into a downstream parser or template, for example $\$(\dots)$ or $\{\{\dots\}\}$, where it can trigger unintended code/function execution, query/template evaluation, call-backs, delays/failures, or crashes. We bound the adversary by a finite raw-artifact edit budget over strings r .

Positive labels. Labels follow capability rather than sender motive: a string is positive when it has executable behavior, reached a downstream parser, and exercised the corresponding harness. Scanner-emitted strings are held to the same standard. Operational effects are reported separately. This matches the risk we care about: a value inert in an ingress rejection log can become actionable when it enters a command runner, automation action, or query template.

Syntactic vs. semantic. CCD is a stateless structural detector for shell, query, template, and policy-language payload structure. Syntactically benign but contextually dangerous abuses such as DNS rebinding use complementary controls.

4. Learning Stable Hostname Rewrites

With the unit and label rule fixed, the next problem is representation. A detector that treats every strange character as suspicious fails immediately at production scale. Once detection is defined over telemetry strings rather than DNS-valid names, the model must ignore rewrites that preserve a payload, but it must not compress away the wide variety of benign data that merely looks malformed.

The distinction is easier to see on the running example. The SQL-delay probe in Figure 1 can be wrapped in service-looking labels, split by delimiters, or encoded and decoded on its way through telemetry while keeping the same query behavior. A benign label containing a quote, brace, or

tracking fragment should not be pulled into that family merely because it shares punctuation. The same pattern appears in shell and template cases: command substitution, $\{\{IFS\}$ spacing, backticks, and quote/comment breakers are different spellings of a small set of downstream actions.

The encoder therefore applies different rewrite operators to benign and positive host artifacts. We call this training scheme Class-Aware Hostname Obfuscation (CAHO), but the idea is simple: benign examples receive the small perturbations produced by logging and instrumentation, while positives also receive attacker-style rewrites that preserve the downstream action. Unlike DNS-only augmentation, these rewrites include forms that survive in telemetry, such as URL-like prefixes, mixed encodings, and delimiter insertion, and that may later be consumed by string-interpolating tooling. Pattern-only models tend to overfit these surface forms [41], [42], [43], [44]; in our deployment, host values containing shell syntax, automation-triggering metacharacters, and SQL template breakers survived logging and reached those tools.

Augmentation sets. Let $\mathcal{T}_0$ contain benign perturbations and $\mathcal{T}_1$ contain positive perturbations, with $\mathcal{T}_1 \supset \mathcal{T}_0$. The benign side receives mild, structure-preserving changes such as case variation, delimiter jitter, and limited token split/merge, matching natural logging variance and instrumentation artifacts. The positive side receives the same benign variation plus attacker-style rewrites that preserve a payload: scheme or prefix insertion (e.g., `https://`, `www.`), percent/UTF-8 encoding and mixed decoding residues, confusable substitution, delimiter injection, comment-style suffixing, and layered escape/substitution patterns. Generated variants that fall outside deployment-accepted host-string formats are discarded.

Design rationale. The contrastive objective enforces two properties: payload-preserving variants should stay close within a family, and benign examples should remain separated from positives [39], [40], [45]. That separation depends on the asymmetric augmentations. Benign perturbations model ordinary logging variance; positive perturbations model the ways an attacker keeps a payload intact while changing its spelling.

If both classes used the same rewrites, the encoder would either treat punctuation as an attack signal or collapse useful benign diversity. Class-aware augmentation ($\mathcal{T}_1 \supset \mathcal{T}_0$) avoids that tradeoff. It trains invariance to attacker-style encodings that survive logging and reach string-interpolating automation, while leaving malformed benign traffic broad. In ablations, this produces compact positive clusters and a broader benign manifold, improving both accuracy and held-out-family behavior [46], [47].

5. Scoring Hostname Neighborhoods

5.1. From Security Intuition to Score

Once equivalent payload spellings stay close, CCD still needs a calibrated decision at the chosen operating point. An

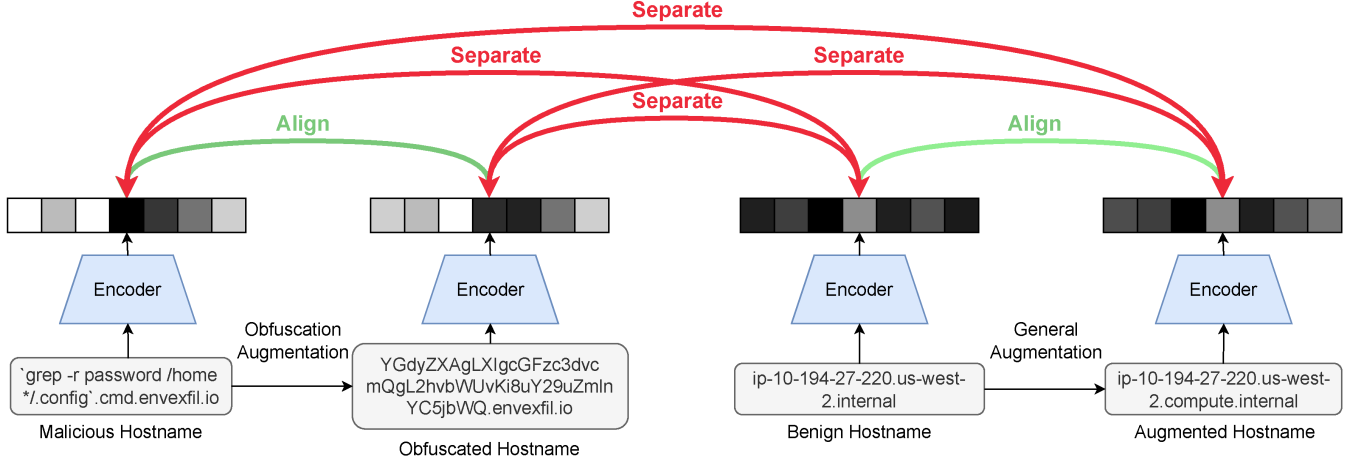


Figure 3. Class-Aware Hostname Obfuscation (CAHO). *Recorded host values* undergo class-aware obfuscation: positive strings receive rich attacker-style transformations; benign strings receive mild perturbations. Training pulls payload-preserving variants together while separating classes. the finite-edit stability verifier. The class-aware training described above pulls obfuscation variants of the same attack class together while preserving benign diversity; the cone sketch then discards exact spellings and retains where the value sits relative to benign and executable regions. Payload-preserving variants concentrate in cones where a positive prior has mass; malformed benign strings remain diffuse around the benign reference. The score gap comes from that separation, not from a memorized token.

Score and decision interface. CCD uses a likelihood-ratio sketch with optional finite-edit decision-stability certificates for retained host artifacts. Given a normalizer Π , versioned edit manifest $\mathcal{E}_{\text{raw}}$, encoder f , cone partition $\mathcal{C}$, benign prior P_B , and optional executable priors $\{P_{M,k}\}$, it exposes three operations: a scalar score $S_\pi(q(r))$, a calibrated decision $h_\alpha(r)$, and an optional certificate $\text{certify}(r, K)$ for the finite edit ball induced by $\mathcal{E}_{\text{raw}}$. Hostnames instantiate this interface with CAHO embeddings and the HIB priors; adjacent-field experiments are scope checks rather than the main claim.

The running example illustrates why the comparison is two-sided. If the Figure 1 string is percent-encoded, label-split, or wrapped in a service-looking prefix, most of its mass should remain near the SQL-delay region. A benign label with a quote or tracking token may share one surface character, but its mass should land in the broader benign background. CCD raises the score only when both facts hold: the value is unusual for ordinary traffic and well explained by at least one executable-semantics class. The effective count n_0 controls how much weight this neighborhood evidence receives; it is selected on validation because both extremes are harmful. If n_0 is too small, the priors wash out, and if it is too large, small sketch movements dominate the score.

The deployed path is fixed before calibration and test: canonicalize the raw artifact with Π , encode it as a unit vector, retrieve nearby angular cones, form the sparse cone sketch $\hat{Q}_u$, score that sketch by Eq. 1, compare it with the tenant’s split-conformal threshold, and optionally run

5.2. Cone Sketches and Cross-Entropy Scoring

The fast path keeps only the information needed for the likelihood test. Conceptually, the statistic is the cone-count distribution used in Eq. 1; the production lookup method only determines how quickly nearby cones are found. Given an input $r \in \mathcal{R}$, we canonicalize to $x = \Pi(r)$ and encode a unit embedding $u = f(x)/\|f(x)\| \in \mathbb{S}^{d-1}$. We partition the unit sphere into $M=4096$ cones indexed by axes $\{\theta_j\}$. The implementation uses multi-probe LSH [48] to retrieve $R \approx 8$ nearby cones in $O(Td + R)$ time rather than scanning all M axes; certification falls back to the exact all-cone score whenever the top- R support is unstable. We then assign fractional mass to the selected cones with a softmax over cosine similarities, producing the sparse sketch $\hat{Q}_u \in \Delta^{M-1}$. The sketch retains where the local neighborhood falls on the sphere, not the spelling that reached the log.

After embedding, cone retrieval and likelihood scoring take 0.04 ms median per item in the production implementation; the full path, including canonicalization, tokenization, encoding, cone retrieval, and scoring, is 0.60 ms amortized per item in the matched replay. The same sketch feeds the fast classifier and the certification path. If certification detects unstable top- R support, deployment recomputes the exact all-cone score for that value before attaching a certificate.

The score compares q with a benign reference P_B and positive references $\{P_{M,k}\}_{k=1}^K$ with mixture weights $\pi_k > 0$ and $\sum_k \pi_k = 1$. For distributions Q, P , cross-entropy $H(Q, P) = -\sum_j Q(j) \log P(j)$ measures how well

CCD on unit sphere with three attack-family priors

$$S(u) = H(\hat{Q}_u, P_B) - \min_k H(\hat{Q}_u, P_{M,k})$$

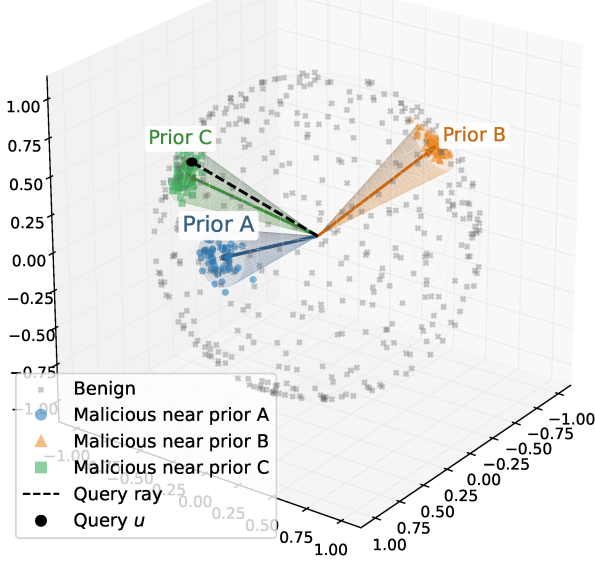


Figure 4. CCD scoring on the unit sphere. The figure projects CAHO embeddings for three payload patterns and benign hostnames. CCD compares how well the benign baseline P_B and the learned positive mixture explain the query’s cone sketch q . If $S_\pi(q) > \tau_\alpha$ (Eq. 1), the hostname is flagged as positive.

P explains Q . Lower cross-entropy means the reference distribution makes the observed summary less surprising.

The score uses a fixed effective count n_0 selected on validation and held constant across all strings and edits. Under the cone-count model, a query with deployed sketch q receives the log-likelihood ratio

$$S_\pi(q) = \log \sum_{k=1}^K \pi_k \exp(n_0[H(q, P_B) - H(q, P_{M,k})]), \quad (1)$$

The exponent compares how surprising q is under benign traffic and under one positive prior. If $P_{M,k}$ explains the sketch better than P_B , the gap is positive; if the sketch looks ordinary for benign traffic, the gap shrinks. The log-sum-exp marginalizes over the unknown class, so Eq. 1 behaves like a soft maximum over families.

The hard best-prior score $n_0[H(q, P_B) - \min_k H(q, P_{M,k})]$ is an ablation and a close approximation when one class contributes most of the mixture likelihood; the deployed score is Eq. 1.

The split is what makes the detector open-set. A string can score highly because it leaves the benign background even when its exact family was not memorized, but the mixture term still favors neighborhoods that resemble executable mechanisms. Constructing $\hat{Q}_u$ touches R cones after LSH; scoring evaluates P_B and K priors only on those R entries, yielding fixed cost and a small state vector.

5.3. Scoring Derivation and Scope

Equation 1 is a likelihood-ratio rule for the sparse summary after the normalizer, encoder, cone partition, and

TABLE 1. DECISION AND STABILITY CONTRACT FOR THE CCD SKETCH. EACH STATEMENT IS SCOPED TO THE RETAINED STATISTIC AND ITS STATED ASSUMPTIONS.

Layer	Contract	Assumption
Cone-count model	Likelihood-ratio test over q	Fixed $\Pi, f, \mathcal{C}, n_0$
Split calibration	Alert/FPR budget τ_α	Benign calibration window
Edit closure	Margin-stable $h_\alpha(r)$ for radius K	Frozen $\mathcal{E}_{\text{raw}}$ and score path

effective count have been fixed on validation data. The statistic asks which distribution better explains the local neighborhood of the query string: benign P_B or a learned positive mixture.

The effective count n_0 controls how much weight CCD gives to this neighborhood evidence. It is fixed by validation rather than by hostname length, so length-changing edits do not change it. This keeps the stability statement tied to the deployed score path and to the retained artifact evaluated by CCD.

The same statistic is used throughout the evaluation. Only the surrounding condition changes: unseen attack classes, adaptive mimicry, temporal and tenant drift, and finally production replay.

Proposition 5.1 (Margin-coupled sketch). *Let $m(r) = S_\pi(q(r)) - \tau_\alpha$ be the deployed score margin. For a fixed normalizer, cone partition, score path, threshold, and edit manifest, suppose all strings z in the declared K -edit ball satisfy $|S_\pi(q(z)) - S_\pi(q(r))| \leq \Delta_K(r)$. Then the positive decision is certified throughout that edit ball whenever $m(r) > \Delta_K(r)$, and the benign decision is certified whenever $-m(r) \geq \Delta_K(r)$. Within the cone-count model, S_π is also the likelihood-ratio statistic in Eq. 1; the same margin therefore measures both detection evidence and edit-stability slack.*

Proof sketch. The decision changes only if an edit moves the score across τ_α . The bound $\Delta_K(r)$ limits the worst score movement over the edit ball, so the sign of $m(r)$ cannot change under the stated inequalities. The likelihood-ratio statement follows from the derivation of Eq. 1 for the retained cone-count statistic. $\square$

Table 1 collects these decision and stability statements with the assumptions that make each one valid.

6. Taxonomy-Guided Synthetic Training

The score needs positive reference distributions, but deployment cannot pool raw tenant payloads to build them. CCD therefore learns from generated positives without letting the generator decide what counts as an attack. That boundary sits in an expert-vetted taxonomy: analysts define the mechanism families, generators propose strings inside those families, and aggregate benign traffic supplies the background structure. This training path lands within 2.4 percentage points of real-payload training at a 10^{-4} false-positive rate (§8.3).

Taxonomy as an interface. We codify hostname injection as families with executable *leaves*. Each leaf describes one downstream behavior and the spellings that

preserve it. The leaf record contains `leaf = {structure, indicators, constraints, ops_budget b}`: a template with payload slots, canonical tokens, deployment parsing requirements over the recorded value, and an edit budget b for semantics-preserving obfuscation in the raw-artifact space.

6.1. Generation Pipeline

Obfuscation families. We generate and evaluate obfuscations with a raw-artifact edit model covering scheme/prefix insertion, confusables, percent/UTF-8 encoding, delimiter toggles, hex/base encodings, and related rewrites. A leaf’s *variant set* contains the recorded strings reachable within its edit budget b . Using one explicit edit model keeps synthesis and evaluation aligned: training sees the classes of evasions the detector must remain stable against when it scores observed telemetry.

Constrained generation with safety gates. For each leaf, analysts provide 3–10 seeds. Constrained generators then propose candidate strings under the leaf grammar. PCFGs and LLMs are interchangeable proposal mechanisms in this process, not labelers. The acceptance path enforces the leaf constraints, rejects malformed or out-of-scope candidates, checks that required indicators and executable semantics are retained, and filters redundant candidates by surface form and structure. Candidates enter the training set only after automated checks and analyst sign-off, yielding a curated frontier of injection payloads per leaf.

The same edit model that generates training neighborhoods also defines the small rewrites over which a deployment should remain stable. After the taxonomy fixes the positive reference, two things can still move: an attacker can change the spelling of one payload, and benign naming patterns can drift over time.

7. Decision Stability and Drift Controls

CCD handles those pressures separately. Finite-edit checks cover small attacker-style edits to the observed value, so a positive decision does not hinge on one spelling. Benign refresh keeps the background model current as tenants add services, scanners, and instrumentation.

Each CCD decision records the raw value, fixed normalization policy, cone sketch, and threshold used at the chosen operating point. Large-margin positives receive a finite-edit decision-stability check over a small typed edit ball containing hostname-relevant punctuation, quoting, encoding, and label edits. The check asks whether every manifest-covered raw-artifact edit within the declared radius remains above threshold; thin-margin cases are still surfaced, but routed to analyst adjudication rather than automatic containment.

The edit model is data-derived rather than hand enumerated. We mine $\mathcal{E}_{\text{raw}}$ from observed normalization differences, related positives with the same executable core, and public payload-corpus variants. The retained families cover scheme/prefix toggling, delimiter insertion or deletion, percent/UTF-8 encoding and decoding, hex/base encodings,

case changes, Unicode confusables, quote/brace/comment fragments, and bounded DNS-label edits. Analysts vet whether a mined operator preserves executable semantics and is safe to certify; they cannot add an unsupported operator only to improve coverage.

The certificate is narrow by design: every edit in the declared finite edit ball must leave the same deployed CCD decision unchanged under the frozen normalizer, cone sketch, score path, threshold, and edit-manifest version. It is not a guarantee about downstream sink safety, unseen parser behavior, or future edit families. If top- R cone support is unstable, CCD recomputes the exact all-cone score before issuing a certificate; randomized edit sampling is used only to find hard cells, and a certificate is emitted only after deterministic edit-ball closure. A later parser or a newly observed transformation family starts a new edit-model version rather than quietly enlarging an old certificate.

Drift uses a different lever. CCD fixes the suspicious cone distribution after validation and refreshes only the benign cone distribution and threshold from clean benign windows. The detector can follow normal naming drift without learning from new suspicious payloads or moving the attack-family reference during an incident. The evaluation keeps the same separation: it establishes the replay and labels, measures accuracy and latency, then tests family holdout, operating-point stability, drift, and live overlap.

8. Experimental Evaluation

We evaluate CCD under the constraints that decide whether it can be deployed: calibrated accuracy at a fixed false-positive budget on the production host-string stream, full-path latency at ingestion, ablations that explain the operating point, robustness under family, time, tenant, public-real, and mimicry shifts, and verified added value in a matched live comparison against maintained regex and WAF controls.

HIB is the production-derived stream in which the detector has to live, not a synthetic payload-only benchmark. Under the resolved-label policy, verified executable-semantics positives are only 0.18% of the replay, and many benign or unresolved rows contain the characters that make a payload look suspicious. A positive label therefore requires executable behavior, a downstream record containing the same value or a reversible encoding, and agreement from the relevant sink-family harness. Analysts reviewed that evidence chain without detector identity.

At this prevalence, calibration is an operational requirement. At $\alpha=10^{-4}$, a small threshold error can send roughly twenty thousand false positives to the queue. We therefore set thresholds before test and run every method on identical captured inputs. Unresolved rows are excluded from replay denominators but remain visible in overlap views.

Evidence units. Table 2 fixes the denominators before the results: replay entries measure accuracy, latency, drift, and baseline comparisons; verified executable-semantics positives measure positive-side replay behavior; live comparison items measure CCD-vs-incumbent overlap; and composite,

TABLE 2. EVIDENCE AND ACCOUNTING UNITS USED IN THE EVALUATION.

Unit	Meaning and use
Replay entry	Denominator for replay accuracy, false-positive rate, latency, drift, and baseline comparisons.
Verified positive	Hostname in the 363,401 replay accounting with executable behavior, downstream evidence, and harness agreement; denominator for positive-side TPR and attack, sink, and obfuscation reporting.
Live comparison item	Denominator for CCD-vs-RegexWAF overlap in the live/shadow comparison.
Composite alert	Correlated response object across detector events; denominator for fixed-volume operational response accounting.
Tenant-visible alert	Composite emitted in the 50-tenant alerting slice with the hostname-injection signal present.
Unresolved	Marker-like string without enough downstream or harness support; excluded from resolved replay denominators but retained in prevalence, drift, and overlap views.

TABLE 3. HIB-PRODUCTION-REPLAY TEST-SET PROFILE FOR CCD-VS-BASELINE REPLAY. RESOLVED ROWS DEFINE REPLAY DENOMINATORS; UNRESOLVED ROWS REMAIN VISIBLE IN PRODUCTION-OVERLAP VIEWS.

Family	Rows	Resolved rows	Unresolved rows
User login hostnames	112,691,188	112,688,728	2,460
DNS-resolution hostnames	87,648,698	87,564,172	84,526
Total	200,339,886	200,252,900	86,986

tenant-visible, and unresolved rows keep response and uncertainty accounting separate.

8.1. Evaluation Test Set and Replay Protocol

Source selection and capture. HIB-Production-Replay covers **2025-02-01T00:00:00Z–2026-02-06T23:59:59Z** and contains two retained-host streams used by the live detector pipeline: **user-login hostname fields** and **DNS-resolution hostname fields**. Each row is a timestamped capture of the production host-string stream at the same ingestion point used by the live detector pipeline.

We record every failed login and failed DNS-resolution attempt because rejection, lookup failure, and parser failure are the paths where malformed host values remain visible. Successful logins and successful DNS resolutions are retained by independent Bernoulli sampling with probability $p=0.001$ to preserve ordinary naming patterns without letting high-volume success traffic dominate the replay. Reported source-volume counts are counts in the recorded replay, and detector comparisons use the same stream without method-specific reweighting.

Table 3 profiles the resulting test set and keeps unresolved rows visible outside replay denominators.

The two sources stress different failure modes: user-login rows dominate benign volume, while DNS-resolution rows dominate malicious-hostname evidence. A detector that only fits one source or one tenant family can therefore look strong in one column and fail the other, so both streams remain in the denominator for every replay claim.

Population and replay integrity. The replay spans **835** tenant identifiers (552 appearing in user-login rows, 833 in DNS rows, and 550 in both). Failed events are complete

and successful events are sampled, so the source streams retain ordinary benign prevalence and tenant-specific class imbalance.

Source-quality repair was small and happened before splitting: two rows lacked content or source time, 2,716 DNS rows required multiline reconstruction, and one tenant identifier was repaired before splitting (Appendix B).

Temporal isolation. HIB is partitioned once into train, validation, calibration, and test windows with tenant-disjoint, time-forward boundaries. CCD and all replayed baselines are trained, selected, and thresholded only on their assigned pre-test partitions. For the production-deployment evaluation, CCD was sealed on **2025-11-01**; training and validation use only data available before that cutoff. Rows recorded from November 2025 through February 2026 remain in replay accounting but are not used to retrain, tune, or re-threshold the model during the deployment window. The same rule applies to learned, LLM, and classical baselines, with no tuning on HIB-test labels or post-window labels. The live alert census is the subset **Nov. 2025–Jan. 2026**; the full HIB production replay spans **Feb. 2025–Feb. 2026**.

Labels. Punctuation alone is not enough for a positive label. Benign traffic contains quotes, delimiter-heavy service names, tracking tokens, scanner artifacts, URL fragments, partial encodings, Unicode/IDNA residues, and tenant conventions; in the replay, 1,047,309 rows contain shell-like metacharacters, and at least 672K DNS rows with such characters are benign or unresolved.

The masked trace packet contains the path evidence without exposing the detector: raw and canonical strings, source family, time bucket, consumer excerpt, consumer family, and harness outcome. Detector identity, score, threshold, and tenant identity were hidden, so the model could not supply its own evidence. A string with injection punctuation but no corroborating consumer record stayed unresolved. A string that appeared in an alert action, query builder, or diagnostic helper and matched the harness outcome could become positive.

Locally hosted LLMs triaged variants and flagged consistency disagreements for human re-check, but never assigned resolved labels. The final replay contains 199,889,499 benign labels, 363,401 verified executable-semantics positives, and 86,986 unresolved labels. Stratified detector-hidden relabeling reached $\kappa \approx 0.97$; Appendix B reports the separate resolved binary replay agreement.

Production-replay profile. All replay and stress-test analyses use captured HIB production traffic with frozen models and preselected thresholds. The 363,401 verified executable-semantics positives are production-derived replay positives; Appendix A reports their attack-family distribution from the same replay. The live comparison is reported separately through live overlap and composite alerts. Table 4 profiles the verified positives used as the replay denominator. This profile prevents interpreting the replay as a single-family signature benchmark; the family-holdout, drift, and mimicry experiments below test generalization more directly.

TABLE 4. VERIFIED-POSITIVE PROFILE IN HIB PRODUCTION REPLAY. THE 363,401 REPLAY POSITIVES DEFINE THE DENOMINATOR.

Profile	Production-replay evidence
Family mix	15 named families plus a 12-example residual; largest: JNDI/Log4Shell-style lookup 58,047 (16.0%), SQL injection 46,623 (12.8%), command injection 37,481 (10.3%), PHP OOB callback 30,097 (8.3%); all remaining named families are at or below 6.9%.
Sink mix	Evidence spans interpreter, SQL/query, shell/command, callback, template/SSTI, URL-fetch/SSRF, XML/XXE, LDAP/NoSQL, path-resolution, and header/CRLF sink classes; sink-family evidence is kept in harness records while the table reports replay-positive family and obfuscation denominators.
Obfuscation	Non-none signals: 264,588 (72.8%); mixed 121,621 (33.5%), delimiter rewrite 55,862 (15.4%), Base64 45,951 (12.6%), percent encoding 33,696 (9.3%), quote/comment 7,458 (2.1%); none 98,813 (27.2%).
Representative examples	See Appendix A: lookup callbacks, encoded SQL delays, shell separators, and runtime-call payloads.

Selection replay versus live queue. The 18-baseline replay is a pre-deployment selection pass. Every replayed candidate is fixed before its evaluated window and run on CCD’s timestamped production stream. Learned and LLM candidates were not inserted into the shared alert queue because their latency, memory, or serving cost was incompatible with the ingestion rate; they remain replay baselines. The live study therefore measures the deployable pair: CCD versus the incumbent RegexUWAF controls.

Static-analysis comparison. Static analyzers operate on repositories rather than HIB telemetry rows, so their on-turf comparison is a source-to-sink corpus of 50 source-available repositories that process host or telemetry fields. A custom CodeQL 2.17.0 taint query found 10 true positives and 2 false positives but missed 38 delayed log/warehouse paths; Semgrep 1.60.0 taint mode found 6 true positives and 2 false positives but missed 42 cross-service telemetry flows. This source-corpus comparison shows why ordinary single-codebase taint analysis is incomplete here: in the missed cases, the source becomes a telemetry record and is read by another automation path.

The misses were not caused by unusual source declarations alone. They were paths where a host-like value was written to a log, warehouse, alert object, or enrichment queue, and a different service read it back to construct a command, query, template, or lookup. Modeling that second-order path manually is possible for a single known application, but it is not the default operating point of either tool. CCD targets the field before that second service uses it.

8.2. Detection Accuracy and Throughput

Figure 5 shows that CCD’s recognition gain is not bought with an unusable service path. Across classical estimators, deep anomaly detectors, and transformer models, CCD has the highest recall at 10^{-4} FPR. Prompt-only and LoRA-fine-tuned local LLMs provide the strongest offline comparison: they use CCD’s splits, validation-selected prompts, deterministic decoding, and a common false-positive target. Appendix B lists the locally served models.

TABLE 5. FIXED-FPR BASELINE AUDIT AT THE PRODUCTION ALERT BUDGET.

Method	TPR@ 10^{-4} (95% CI)	FPR	p99/p99.9 ms	Queue role
CCD	0.935 [0.929, 0.941]	9.8×10^{-5}	3.92 / 7.80	Inline, live queue budget
DeBERTa-v3-small head	0.896 [0.888, 0.904]	9.9×10^{-5}	28.70 / 55.30	Production replay
URLBERT tuned head	0.881 [0.872, 0.889]	1.01×10^{-4}	17.80 / 34.40	Production replay
Qwen-family LoRA LLM	0.851 [0.833, 0.869]	9.8×10^{-5}	294 / 548	Async triage only
Handcrafted GBDT	0.782 [0.768, 0.795]	9.6×10^{-5}	2.70 / 5.90	Fast, lower recall
Regex/WAF taxonomy	0.704 [0.688, 0.720]	9.7×10^{-5}	1.30 / 3.10	Incumbent control

CCD keeps the recognition gain in a service that can run where host strings arrive.

For inline enforcement, the full path matters more than the scoring kernel alone. In a 64-concurrent, batch-1 replay, CCD’s end-to-end tails are 1.84/3.92/7.80 ms p95/p99/p99.9 and one host processes 48k host strings/s. After embedding, cone retrieval and likelihood scoring measure 0.04 ms p50 and 0.11 ms p99. Larger 10^5 – 10^6 host-string/s deployments use the same service behind ordinary ingestion sharding. Locally run LLM baselines are typically 10^2 – $10^3 \times$ slower on this hardware, while the non-LLM methods that approach CCD’s latency lose accuracy at the production alert budget. Table 5 keeps the comparison on matched splits, validation-only model selection, benign split-conformal thresholds, tenant-block bootstrap intervals, and full-path p99/p99.9 latency unless noted.

Table 5 separates two questions that Figure 5 compresses visually. Compact transformer and LLM classifiers are serious replay baselines, but their latency tails fit offline triage rather than the ingestion path. Fast engineered and signature baselines fit the queue, but lose the behavioral recall that CCD preserves at the same false-positive budget.

Robustness to unseen attack classes. A harder test removes an obfuscation family from synthetic priors and encoder training, then measures whether the unchanged detector still recognizes that mechanism at the same alert budgets. We run this leave-one-family-out experiment with thresholds set on benign-only holdouts and applied once at three fixed false-positive rates.

CCD has the highest recall across held-out families (Figure 6). Against the strongest non-CCD baseline (CSI), it gains 6–8 points at each alert budget. Since the withheld class is absent from synthetic priors and encoder training, the result points to the benign-divergence term $H(\hat{Q}_u, P_B)$ in Eq. 1: unseen mechanisms still move away from the benign cone distribution.

A primitive operator is one shell or URL construct such as backticks or subshell. Composition depth d counts primitive occurrences in the hostname. We train priors and CCD on seen mechanisms with $d \leq 2$, then freeze thresholds to target $\text{FPR} = 10^{-4}$. We perform two tests without re-tuning: zero-shot mechanisms, where entire operator classes are withheld, and depth extrapolation, where mechanisms are seen but evaluated at $d \in \{3, 4, 5\}$.

At the 10^{-4} false-positive point, zero-shot depth-3 recall ranges from 0.74 to 0.83 and remains 0.59 to 0.71 at depth 5. When the mechanism is seen but the composition is deeper, macro recall drops from 0.92 ($d \leq 2$) to 0.83 at depth 5, with every mechanism at least 0.79.

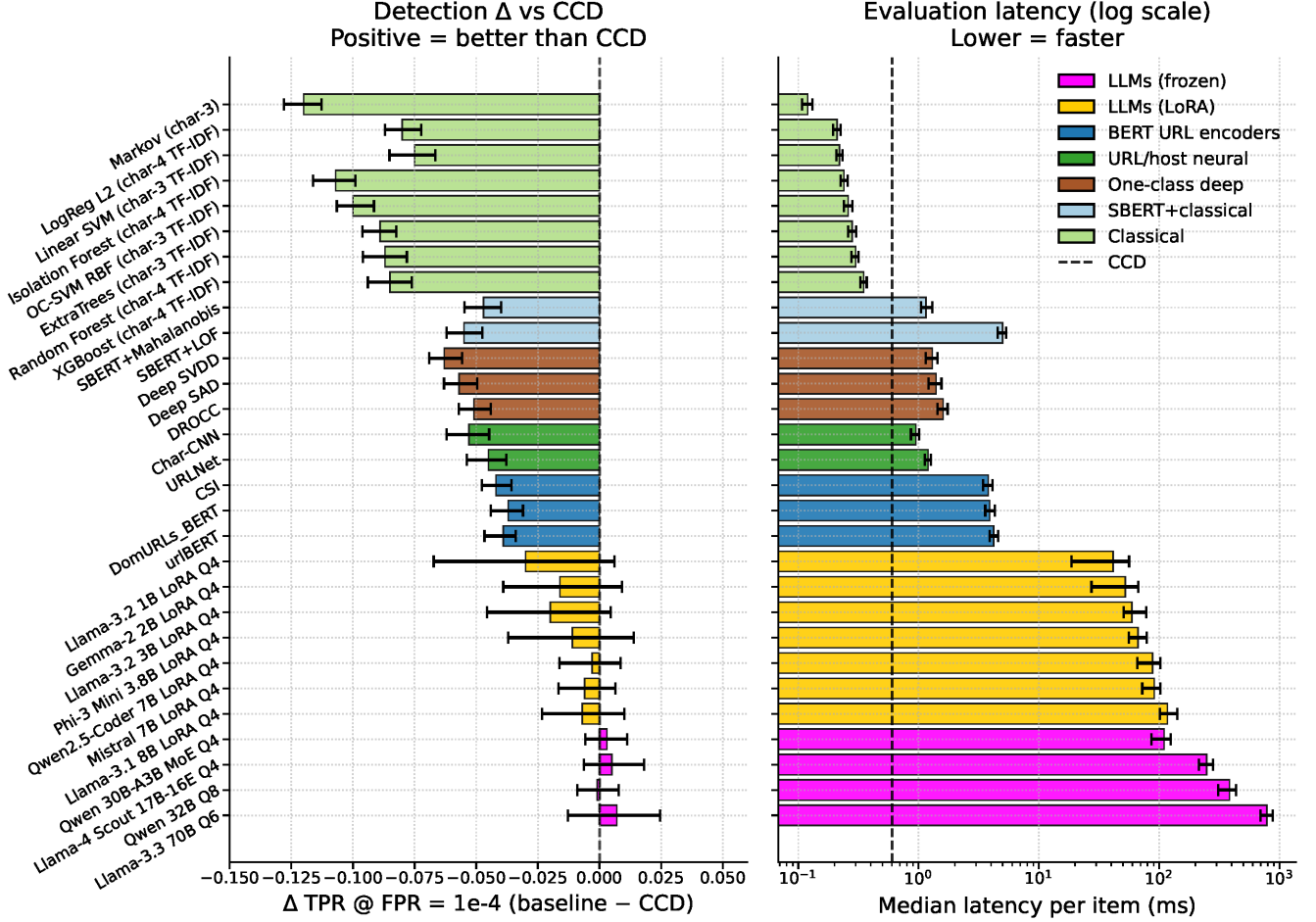


Figure 5. Accuracy and latency at $\text{FPR} = 10^{-4}$. **Left:** ΔTPR relative to CCD by method family. **Right:** median per-item latency on the same Ryzen host, including feature extraction, encoding, and classification. CCD’s 0.60 ms is amortized full-path batch-1 replay latency. Error bars: 95% bootstrap CIs across tenants ($n=835$).

8.3. Calibration, Ablations, and Drift

We first check that thresholding behaves like an analyst queue rather than a nominal score cutoff. CCD sets τ_α per tenant from benign holdout scores with split-conformal quantiles and the baseline budget. Across $\alpha \in [10^{-4}, 10^{-2}]$, realized false-positive rates track the target; at the reported budgets, the mixture score and hard best-prior ablation are statistically indistinguishable.

Why the obvious integration fails. A natural baseline is a contrastive encoder with a conventional head, calibrated threshold, and post-hoc robustness checks. Table 6 shows why that composition is not the CCD score-and-sketch design. On ordinary replay rows, learned geometry and the benign-divergence term provide most recall; under adaptive pressure, the representation also has to keep payloads away from benign-looking regions when the spelling is chosen to camouflage them.

The white-box column makes the role of the score clear. Removing cone sketching costs 8.9 points on the main replay but 37.4 points against a white-box attacker. Payload-variant alignment and class-aware augmentation isolate the

TABLE 6. MECHANISM ABLATION UNDER MATCHED SPLITS AND FIXED FALSE-POSITIVE THRESHOLDS. VALUES ARE RECALL-POINT DROPS FROM FULL CCD.

Variant removed	HIB 10^{-4}	HIB 10^{-5}	Holdout	White-box
None: full CCD reference	0.0	0.0	0.0	0.0
Linear head instead of cone sketch	-8.9	-19.0	-26.8	-37.4
No payload-variant alignment	-1.2	-2.7	-3.7	-19.1
No class-aware positive augmentation	-2.3	-4.8	-10.6	-13.3
No conformal calibration	-4.2	-10.1	-5.6	-9.2

contrastive-training choices; the no-conformal row isolates fixed-FPR calibration; and the generator comparison below isolates the taxonomy/source axis. CAHO builds the neighborhoods, and the cone likelihood ratio makes them hard to camouflage as benign traffic.

The generator comparison measures the cost of avoiding raw tenant positives. PCFG-only priors reach 85.2%, LLM priors 93.5%, and real-payload training 95.9% at the production threshold. Grammar captures coarse mechanism structure; LLM proposals add diversity; real payloads add modest signal. The remaining gap is much smaller than the adaptive margin supplied by the cone scorer in Table 6.

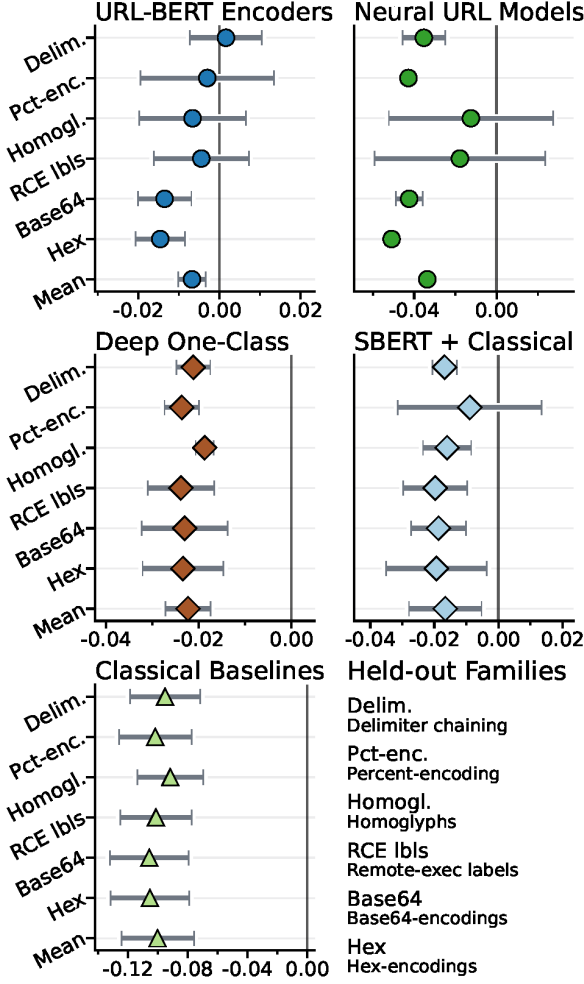


Figure 6. Leave-one-family-out performance at $\text{FPR} = 10^{-3}$. Each cell reports the strongest baseline in its non-LLM family minus CCD on the held-out family; negative values indicate higher CCD TPR.

Decision stability. The check answers a narrower deployment question: when CCD fires, should small rewrites of the observed string flip that one decision? Under the fixed edit model, CCD reports a decision-stability certificate rather than a claim about downstream safety. It combines calibrated-margin certificates (CMC), which prove that the deployed score stays on the same side of τ_α over the finite edit ball, with smoothed-search enumeration certificates (SEC), which accept only after deterministic edit-ball closure. At $K=6$, the combined check reports stable decisions for 98% of detected positives (Figure 7). The certificate binds to the deployed normalizer, sketch, score, threshold, and edit-manifest version.

For drift, CCD lets only the background model move. Tenant naming patterns change continuously, while the positive reference should not chase an incident. Refreshing (P_B, τ_α) on recent benign windows keeps recall near-flat through five months of temporal shift and gives the slowest observed degradation under cross-tenant divergence. It does so without updating encoder parameters or collecting new positive labels. On the evaluated hardware, refreshing a two-

week benign window took 40 s, compared with hour-scale retraining paths for neural URL baselines.

The operating point is reproducible beyond the operator’s raw telemetry. An independent replay of release d2a5b6 over the released de-identified evaluation rows matches deterministic row and count metrics within 0.01%, and three blind tenant holdouts keep realized false-positive rates within $\pm 0.2 \times 10^{-4}$ of target.

A public-real scope check gives an external version of the same test. On public-real positives, synthetic-only training reaches 87.3% recall at the 10^{-4} operating point; public-real training reaches 91.1%; adding 5% public-real data to the synthetic set reaches 92.8%. Real payloads carry extra signal, but the synthetic model remains close enough for deployment. Public reports also mark the taxonomy boundary: 118 of 127 post-freeze reports map to target classes, with the nine exclusions being rebinding, certificate-validation, or QUIC cases.

Hostile mimicry stresses a different failure mode: the payload remains executable while its spelling moves toward benign-looking regions. In the white-box setting, the attacker receives the encoder, cone partition, score, normalizer, and threshold. Across marker-preserving, marker-erasing, cone-escape, DNS-segmentation, and IDNA suites, CCD keeps 91.5% to 99.2% recall at 10^{-4} FPR; the strongest baselines range from 45.8% to 85.4%. The live comparison then tests whether those margins translate into additional verified cases without expanding the queue.

8.4. Production Deployment Results

After the 835-tenant replay selected CCD as the deployable model, we compared it with the maintained regex and WAF controls on the Nov. 2025–Jan. 2026 live/shadow hostname stream. The matched live comparison asks which verified retained-host case chains reached detector-hidden review under the same evidence-bearing composite-alert workload. A *live comparison item* is the retained-host case chain used for detector overlap; a *composite alert* is the correlated response object that may contain evidence from several detectors; and a *tenant-facing alert* is a composite emitted in the 50-tenant alerting slice. These categories are separate, so the overlap counts below are neither new-alert counts nor production-compromise claims.

The live review reused the masked trace packet from HIB labeling. A verified item required the raw string, a downstream record carrying the same value or a reversible encoding, the pipeline stage, coarse success/error bucket, sink family, and harness outcome; detector identity and confidence were hidden. Independent live-stratum relabeling reached $\kappa \approx 0.97$; Appendix B separately reports binary replay agreement. The same detector-hidden rubric anchors the production-replay malicious-hostname labels.

Evidence tiers. The review keeps mechanism evidence separate from response outcomes. *Callback* means DNS or HTTP callback behavior in a matching harness; *delay/failure* covers query or command delay, compile error, crash, or nonzero failure caused by the string; *template expansion*

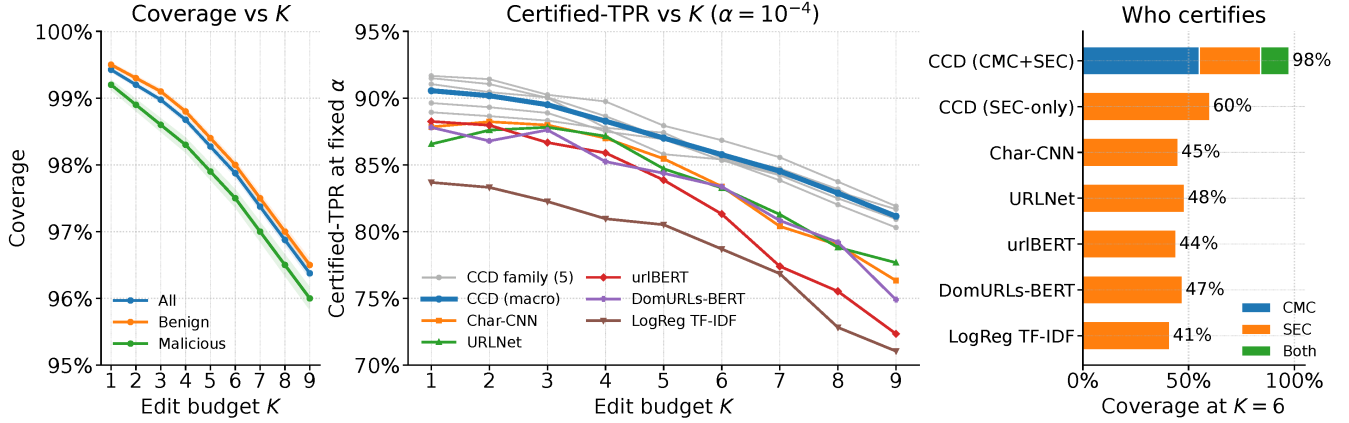


Figure 7. Finite-edit decision-stability at $\alpha=10^{-4}$ with τ_α calibrated on benign and frozen across edit budgets K . Left: stable-decision coverage for all, benign, and verified-positive points. Middle: stable-decision TPR by payload family. Right: at $K=6$, CCD reports stable decisions for 98% of detected verified positives (SEC alone: 60%) versus 41–48% for native-feature baselines.

covers macro or template evaluation; and *blocked side effect* records a payload reaching the sink with the effect disabled or denied. *Unresolved* strings have marker-like structure without enough downstream support. *Tenant-escalated* is a response-level outcome for a correlated alert, not a row label.

Live overlap. Table 7 shows the matched workload over retained case-chain records with input-parity checks, configuration/runtime identifiers, masked-review summaries, and harness identifiers. CCD and RegexWAF agreed on **2,300** verified positives, mostly established payload forms. The production difference is not a larger queue, but the remaining **850** CCD-only verified live-comparison positives: each reappeared in downstream evidence, matched a sink outcome, and had no incumbent hit. Across the window, CCD flagged **3,150** verified live positives, including **1,740** obfuscated variants. The same signal propagated to **315** composite alerts, compared with **147** for RegexWAF.

TABLE 7. MATCHED LIVE OVERLAP. LOWER ROWS RETAIN BASELINE-ONLY UNCERTAIN AND ALL-ALERT ACTIVITY.

	CCD $\cap$ (RUW)	CCD only	(RUW) only
Verified positives	2,300	850	0
Verified + uncertain	2,390	855	90
All alerts (any label)	2,500	855	200

The incumbent side remains visible in the accounting. RegexWAF produced 200 baseline-only all-alert items; 90 remained uncertain after masked review, 110 were verified benign under the same rubric, and none reached verified-positive status. The zero in the verified row therefore means CCD’s verified live set covered the incumbent verified live set under an identical detector-hidden label rule; it does not mean the incumbents emitted no alerts.

The reviewed-item denominator also fixes the precision and false-positive interpretation. Counting uncertain items as non-positive, CCD-bearing live comparison items had a lower-bound reviewed-item PPV (verified yield)

of $3150/3355=93.9\%$ and RegexWAF-bearing items had $2300/2700=85.2\%$. Over the 92-day window, verified-benign review load was 1.2 items/day for CCD-bearing items and 2.4 items/day for RegexWAF-bearing items; counting uncertain as non-positive gives 2.2 and 4.3 non-verified items/day, respectively. These are reviewed live-comparison rates, not global stream precision.

Tenant-facing alerts. Tenant-facing alerts were enabled for a limited slice of 50 tenants; tenants outside this slice ran in replay/shadow mode. These higher-level objects correlate related events across detectors to reduce alert fatigue. Within the slice, CCD-bearing composites produced **139** tenant-visible alerts with the injection signal present. Two organizations independently escalated CCD reports as attack attempts, while others requested guidance or additional context.

What CCD found beyond the baselines. The 850 CCD-only verified live positives are a fixed-volume live-overlap delta, not tenant-visible alerts. They came from replayable payload families rather than punctuation edge cases: command-substitution callbacks toward diagnostic helpers, brace/quote/template fragments in alert-enrichment labels, SQL quote/comment breakers in warehouse dimensions, double-decoding and $\{\text{IFS}\}$ bypasses, OAST callbacks hidden under internal-looking suffixes, and encoded variants missing fixed regex/WAF tokens. The evidence tiers included callback behavior, delay/failure, template expansion, and blocked side effects, with unresolved strings kept out of the verified-positive row in Table 7. The added coverage therefore comes from command, template, query, and alert-action contexts rather than from a threshold shift on one attack class or a single JNDI lookup family.

8.4.1. Case-study replay: execution and operational effects. We replayed three recorded strings through the frozen production CCD, regex, and WAF configurations. Each retained raw value, later exact/reversible record, sink family, harness behavior, and response evidence; marker-only strings remained benign or uncertain.

Replay outcomes. Table 8 reports the parser, persistence, consumer, controlled effect, and detector boundary used for adjudication. The table is an evidence trace, not a compromise claim: effects were reproduced in controlled matching-sink replay, with side effects blocked or restricted to researcher-controlled endpoints.

TABLE 8. END-TO-END EVIDENCE TRACES FOR AUDITED METADATA-TO-CODE PATHS. THESE ARE CONTROLLED REPLAY EFFECTS, NOT COMPROMISE CLAIMS.

Case	Observed chain and controlled sink effect
Diagnostic helper	A command-substitution host-like value <code>\$(nslookup -q=cname OAST curl OAST)</code> under an internal-looking suffix failed downstream hostname parsing but persisted in a host-dimension field. A legacy “diagnose host” action interpolated that field into a DNS/HTTP helper; controlled replay produced DNS/HTTP callbacks from the investigation-service environment, and CCD fired at the telemetry boundary before helper consumption.
Alert action	A quote/brace template fragment persisted into an alert-enrichment label and was later rendered by an alert action. Controlled replay produced macro/template expansion and reached a side-effect guard with the action disabled; the case supports template-reuse reachability rather than production effect exploitation.
Analytics query	A SQL quote/comment breaker survived as a warehouse dimension and entered a scheduled templated query. Controlled replay produced the expected delay/failure behavior, including a bounded query error rather than data modification; CCD scored the retained value before the query builder consumed it.

Downstream signal. Of the **3,150 verified live positives**, **92%** carry RFC 1123-violating characters (e.g., `|`, `$`, `{`, `}`) as recorded downstream. In other words, RFC-oriented ingress enforcement did not prevent payload-bearing hostnames from remaining available to telemetry consumers. This live statistic is separate from the 363,401 replay attack-vector accounting.

Implications. At the matched alert volume, CCD reduces reliance on continual rule expansion while surfacing obfuscated positives that evade brittle signatures. The evaluated set spans CWE-78/89/94/1336-style sink classes rather than a single bug pattern. The taxonomy also covers public evidence outside the deployment: 118 of 127 ($\approx 93\%$) post-freeze public CVEs map to target categories, while the exclusions are rebinding, certificate-validation, or QUIC cases that require complementary controls. Downstream command, query, template, URL-fetch, and alerting systems still need parameterization, escaping, and least-privilege sink design.

Appendix E and Table 11 summarize which replay, baseline, live-overlap, and sink-evidence claims are released or recomputable, and which production details remain private by design.

Limitations and scope. HIB is one privacy-preserving deployment, and its prevalence does not establish a universal base rate. We use it for what only a deployment stream can

answer: whether a detector can run at production prevalence, hold a calibrated queue budget, and add verified cases at the incumbent review volume. The public-real replay, 50-repo source corpus, and public-CVE coverage probe the same patterns outside that deployment, but they do not cover every organization’s sink mix or malformed benign traffic. In the live stream, the incumbent-only alerts in Table 7 remained benign or uncertain under the same label policy used for CCD and overlap items. Certificates cover only the fixed deployed configuration and finite edit ball; downstream parsers require ordinary hardening and, if needed, a new edit-model version.

9. Conclusion

Retained metadata can outlive the parser that rejected it. In the systems we studied, host fields that were invalid or harmless for routing later became durable input to diagnostic, query, template, and alerting code. That handoff is a real security boundary, even when the original network request has failed.

CCD places the control at that handoff. Across a 200.3M-row, 835-tenant replay, it reached 93.5% TPR at 10^{-4} FPR with 0.60 ms full-path amortized latency. In the matched live comparison, it surfaced verified downstream positives, including 850 missed by deployed regex and WAF controls under detector-hidden review. The broader lesson is that rejected operational metadata deserves a final check before another system turns it into code.

References

- [1] J. Chen, J. Jiang, H. Duan, N. Weaver, T. Wan, and V. Paxson, “Host of troubles: Multiple host ambiguities in http implementations,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS ’16)*. ACM, 2016, pp. 1687–1698. [Online]. Available: <https://dl.acm.org/doi/10.1145/2976749.2978394>
- [2] D. Fett, R. Küsters, and G. Schmitz, “A comprehensive formal security analysis of oauth 2.0,” in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 1204–1215. [Online]. Available: <https://doi.org/10.1145/2976749.2978385>
- [3] D. Fett, R. Küsters, and G. Schmitz, “The web sso standard openid connect: In-depth formal security analysis and security guidelines,” in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*, 2017, pp. 1–15. [Online]. Available: <https://doi.org/10.1109/CSF.2017.20>
- [4] L. Innocenti, R. Pelliccio, M. Squarcina, M. Giannini, A. Continella, C. Verre, T. Pecorella, A. Bianchi, A. Lanzi, and A. De Benedictis, “OAuth redirect uri validation falls short, literally,” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2023, pp. 2473–2487. [Online]. Available: <https://doi.org/10.1145/3627106.3627140>
- [5] C. Jackson, A. Barth, A. Bortz, W. Shao, and D. Boneh, “Protecting browsers from dns rebinding attacks,” *ACM Transactions on the Web*, vol. 3, no. 1, 2009. [Online]. Available: <https://doi.org/10.1145/1462148.1462150>
- [6] M. Johns, S. Lekies, and B. Stock, “Eradicating dns rebinding with the extended same-origin policy,” in *Proceedings of the 22nd USENIX Security Symposium (USENIX Security ’13)*, 2013, pp. 621–636. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/johns>

- [7] D. Liu, S. Hao, and H. Wang, "All your dns records point to us: Understanding the security threats of dangling dns records," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 1414–1425. [Online]. Available: <https://doi.org/10.1145/2976749.2978387>
- [8] P. Jeitner and H. Shulman, "Injection attacks reloaded: Tunnelling malicious payloads over dns," in *30th USENIX Security Symposium (USENIX Security '21)*, 2021, pp. 3165–3182. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity21/presentation/jeitner>
- [9] "NVD advisory: OpenSSH hostname/user expansion OS command injection," National Vulnerability Database (NVD), 2023.
- [10] "NVD advisory: Dropbear dbclient command injection via untrusted hostname argument," National Vulnerability Database (NVD), 2025.
- [11] "NVD advisory: Argument injection in Mitel 6800/6900 series SIP phones," National Vulnerability Database (NVD), 2024.
- [12] "NVD advisory: Command injection in Dasan GPON home routers," National Vulnerability Database (NVD), 2018.
- [13] "NVD advisory: NETGEAR DGN2200 ping.cgi OS command injection," National Vulnerability Database (NVD), 2017.
- [14] "NVD advisory: TP-Link Archer AX21 unauthenticated command injection," National Vulnerability Database (NVD), 2023.
- [15] "NVD advisory: TamronOS /api/ping OS command injection," National Vulnerability Database (NVD), 2023.
- [16] "RHSA-2026:7381: Cockpit remote code execution vulnerability," Red Hat Customer Portal Security Advisory, 2026. [Online]. Available: <https://access.redhat.com/errata/RHSA-2026:7381>
- [17] "Securing the git push pipeline: Responding to a critical remote code execution vulnerability," GitHub Blog, 2026. [Online]. Available: <https://github.blog/security/securing-the-git-push-pipeline-responding-to-a-critical-remote-code-execution-vulnerability/>
- [18] "Host header injection via ctx.hostname," koajs/koa GitHub Security Advisory GHSA-7gcc-r8m5-44qm, 2026. [Online]. Available: <https://github.com/koajs/koa/security/advisories/GHSA-7gcc-r8m5-44qm>
- [19] T. Galloway, K. Karakolios, Z. Ma, R. Perdisci, A. D. Keromytis, and M. Antonakakis, "Practical attacks against DNS reputation systems," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024, pp. 4516–4534. [Online]. Available: <https://doi.org/10.1109/SP54263.2024.00266>
- [20] X. Li, W. Xu, B. Liu, M. Zhang, Z. Li, J. Zhang, D. Chang, X. Zheng, C. Wang, J. Chen, H. Duan, and Q. Li, "TuDoor Attack: Systematically exploring and exploiting logic vulnerabilities in DNS response pre-processing with malformed packets," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://ieeexplore.ieee.org/document/10646751>
- [21] D. Liu, R. Duan, and J. Wang, "Resolution without dissent: In-path per-query sanitization to defeat surreptitious communication over DNS," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025, pp. 1808–1824. [Online]. Available: <https://doi.org/10.1109/SP61157.2025.00004>
- [22] L. Bilge, E. Kirda, C. Kruegel, and M. Balduzzi, "EXPOSURE: Finding malicious domains using passive DNS analysis," in *Network and Distributed System Security Symposium (NDSS)*, 2011. [Online]. Available: https://sites.cs.ucsb.edu/~chris/research/doc/ndss11_exposure.pdf
- [23] F. Deniz, M. Nabeel, T. Yu, and I. Khalil, "MANTIS: Detection of zero-day malicious domains leveraging low reputed hosting infrastructure," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025. [Online]. Available: <https://arxiv.org/abs/2502.09788>
- [24] Q. Wang, J. Chen, Z. Jiang, R. Guo, X. Liu, C. Zhang, and H. Duan, "Break the wall from bottom: Automated discovery of protocol-level evasion vulnerabilities in web application firewalls," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://www.jianjunchen.com/p/wafmanis.sp24.pdf>
- [25] E. Wang, J. Chen, W. Xie, C. Wang, Y. Gao, Z. Wang, H. Duan, Y. Liu, and B. Wang, "Where URLs become weapons: Automated discovery of SSRF vulnerabilities in web applications," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://www.jianjunchen.com/p/ssrfuzz.sp24.pdf>
- [26] R. Yang, H. Wang, Z. Sun, Z. Liu, and Y. Cao, "Poisoned by the host: Large-scale measurement of host name poisoning in web applications," in *2026 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2026. [Online]. Available: <https://www.iamruiyang.me/papers/sp26-HNP.pdf>
- [27] H. Le, Q. Pham, D. Sahoo, and S. C. H. Hoi, "URLNet: Learning a URL representation with deep learning for malicious URL detection," *arXiv preprint arXiv:1802.03162*, 2018. [Online]. Available: <https://arxiv.org/abs/1802.03162>
- [28] Y. Li, Y. Liu, P. Li, Y. Jia, and Y. Wang, "Continuous multi-task pre-training for malicious url detection and webpage classification," *arXiv preprint arXiv:2402.11495*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.11495>
- [29] J. Tack, S. Mo, C. Jeong, and J. Shin, "Csi: Novelty detection via contrastive learning on distributionally shifted instances," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5112b86b7d7b9eb7e94c9e8b4717a9-Paper.pdf
- [30] M. U. Rehman, H. Ahmadi, and W. U. Hassan, "FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://dartlab.org/assets/pdf/flash.pdf>
- [31] Z. Cheng, Q. Lv, J. Liang, Y. Wang, D. Sun, T. Pasquier, and X. Han, "KAiROS: Practical intrusion detection and investigation using whole-system provenance," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://www.vanbastelaer.com/publication/kairos/kairos.pdf>
- [32] A. Goyal, G. Wang, and A. Bates, "R-CAID: Embedding root cause analysis within provenance-based intrusion detection," in *2024 IEEE Symposium on Security and Privacy (SP)*, 2024. [Online]. Available: <https://gangw.cs.illinois.edu/rcaid-sp24.pdf>
- [33] M. Du, F. Li, G. Zheng, and V. Srikanth, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017, pp. 1285–1298. [Online]. Available: <https://users.cs.utah.edu/~lifeifei/papers/deeplog.pdf>
- [34] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," *arXiv preprint arXiv:2103.04475*, 2021. [Online]. Available: <https://arxiv.org/abs/2103.04475>
- [35] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *2025 IEEE Symposium on Security and Privacy (SP)*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.11358>
- [36] Y. Jia, Y. Liu, Z. Shao, J. Jia, and N. Gong, "PromptLocate: Localizing prompt injection attacks," *arXiv preprint arXiv:2510.12252*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.12252>
- [37] Y. Kaya, A. Landerer, S. Pletinckx, M. Zimmermann, C. Kruegel, and G. Vigna, "When AI meets the web: Prompt injection risks in third-party AI chatbot plugins," *arXiv preprint arXiv:2511.05797*, 2025. [Online]. Available: <https://arxiv.org/abs/2511.05797>
- [38] J. Cohen, E. Rosenfeld, and J. Z. Kolter, "Certified adversarial robustness via randomized smoothing," in *Proceedings of the 36th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 1310–1320. [Online]. Available: <https://proceedings.mlr.press/v97/cohen19c.html>
- [39] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 1597–1607. [Online]. Available: <https://proceedings.mlr.press/v119/chen20j.html>

- [40] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, “Supervised contrastive learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: <https://arxiv.org/abs/2004.11362>
- [41] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, “Intriguing properties of neural networks,” in *International Conference on Learning Representations (ICLR)*, 2014. [Online]. Available: <https://arxiv.org/abs/1312.6199>
- [42] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and harnessing adversarial examples,” in *International Conference on Learning Representations (ICLR)*, 2015. [Online]. Available: <https://arxiv.org/abs/1412.6572>
- [43] W. Xu, Y. Qi, and D. Evans, “Automatically evading classifiers: A case study on pdf malware classifiers,” in *Network and Distributed System Security Symposium (NDSS)*, 2016. [Online]. Available: <https://evademi.org/docs/evademi.pdf>
- [44] J. D. Likarish, E. Jung, and I. Jo, “Obfuscated malicious javascript detection using classification techniques,” in *Proceedings of the 4th International Conference on Malicious and Unwanted Software (MALWARE)*, 2009, pp. 47–54. [Online]. Available: <https://ieeexplore.ieee.org/document/5403020>
- [45] J. Robinson, C.-Y. Chuang, S. Sra, and S. Jegelka, “Contrastive learning with hard negative samples,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://arxiv.org/abs/2010.04592>
- [46] T. Wang and P. Isola, “Understanding contrastive representation learning through alignment and uniformity on the hypersphere,” in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 119, 2020, pp. 9929–9939. [Online]. Available: <https://proceedings.mlr.press/v119/wang20k.html>
- [47] J. Tack, S. Mo, C. Jeong, and J. Shin, “Csi: Novelty detection via contrastive learning on distributionally shifted instances,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/4c5112b86b7d7b9eb7e94c9e8b4717a9-Paper.pdf
- [48] Q. Lv, W. Josephson, Z. Wang, M. Charikar, and K. Li, “Intelligent probing for locality sensitive hashing: Multi-probe lsh and beyond,” *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 1177–1188, 2007. [Online]. Available: <https://doi.org/10.14778/3137765.3137836>

Appendix A.

Case Studies and Sink Evidence

Deployment-observed case-study pattern. The three case classes used during adjudication were selected before model comparison and cover distinct downstream sinks. In the shell-diagnostics class, a rejected host-like value containing command-substitution syntax persisted in telemetry and later reached an operator helper that constructed a shell command for lookup; the matching harness observed callback behavior. In the alert-action class, quote, brace, and delimiter fragments in host labels reached an automated enrichment path that performed template or macro expansion; the production response rotated relevant credentials as a precaution, while finding no persistence evidence. In the analytics class, SQL quote/comment breakers reached dashboard or scheduled-report templates through hostname fields and caused query failures or job disruption in warehouse logs. In all three, routing or lookup failed first, telemetry preserved the value, a later tool consumed it under a different grammar, and CCD fired before that reuse.

The three classes differ in where the logged value becomes code. Shell-diagnostics reuse sends a host field to a command runner; alert-action reuse interpolates host labels into enrichment templates or lookup macros; analytics reuse passes hostname fields into query templates. Together, they cover command, template/macro, and SQL/query sinks while keeping operational response separate from detector correctness.

Attack-vector composition. The production-replay HIB experiments use 363,401 verified executable-semantics positives. Table 9 reports their attack-family distribution. JNDI remains the largest HIB family at 16.0%, SQL injection is 12.8%, and no family exceeds 16%. Across the same positives, 264,588 (72.8%) carry a coarse obfuscation signal beyond *none*: mixed signals account for 121,621 (33.5%), delimiter rewrites for 55,862 (15.4%), Base64 for 45,951 (12.6%), percent encoding for 33,696 (9.3%), and quote/comment patterns for 7,458 (2.1%).

TABLE 9. ATTACK-FAMILY DISTRIBUTION FOR VERIFIED PRODUCTION-REPLAY POSITIVES. SHARES USE 363,401 AS THE DENOMINATOR.

Attack family	Count	Share
JNDI lookup	58,047	16.0%
SQL injection	46,623	12.8%
Command injection	37,481	10.3%
PHP OOB callback	30,097	8.3%
Perl OOB callback	25,013	6.9%
XSS	22,020	6.1%
CRLF injection	19,991	5.5%
SSRF URL	19,093	5.3%
Code evaluation	18,967	5.2%
SSTI	17,029	4.7%
XXE	16,011	4.4%
Path traversal	15,033	4.1%
OOB DNS exfiltration	14,053	3.9%
LDAP injection	12,011	3.3%
NoSQL injection	11,920	3.3%
Other	12	< 0.1%

Representative strings. The replay examples span nested lookup callbacks, encoded SQL delays, double-encoded shell separators, `${IFS}` command chains, Python import expressions, and OGNL/Struts-style runtime calls. Released examples preserve delimiters, encodings, casing, and character classes needed to test the detector without exposing tenant-identifying strings.

Appendix B.

Dataset Profile and Splits

HIB replay and splits. HIB contains 200,339,886 privacy-preserving production replay rows from 835 tenant identifiers over 12 months. Under the conservative label policy used for main metrics, the replay contains 199,889,499 benign labels, 363,401 positives, and 86,986 unresolved labels. Its partition is tenant-disjoint and time-forward: training and validation precede calibration, calibration thresholds are fit

only from benign rows, and test windows are never used for threshold selection.

Dataset profile. The replay is split across 112,691,188 user-login rows (56.25%) and 87,648,698 DNS-hostname rows (43.75%). It records every failed login and failed DNS-resolution attempt, while successful events are Bernoulli sampled; the retained rows include 109,973,571 successful logins, 2,717,616 failed logins, 50,904,251 successful DNS resolutions, and 36,744,446 failed DNS resolutions. Binary benign/malicious agreement excluding unresolved rows is 99.98% observed agreement, with Cohen’s $\kappa=0.9585$ overall; DNS is the harder source with $\kappa=0.9546$. Only two rows lacked content or source time, 2,716 DNS rows needed multiline repair, and one tenant identifier was repaired before splitting.

Released benchmark. The de-identified benchmark package is available at https://anonymous.4open.science/r/hostname_command_injection. It contains the complete HIB-Real evaluation replay after de-identification, not a sampled summary or synthetic substitute: all 200,339,886 rows used for the paper’s replay metrics, including resolved benign rows, verified executable-semantics positives, and unresolved rows kept as a separate state. This completeness claim is scoped to the retained HIB replay after the source-capture policy above; live-deployment claims use masked case-chain summaries and harness outcomes rather than raw tenant logs. Records carry stable identifiers, privacy-preserving raw and canonical strings, source family, coarse time bucket, resolved label, label-support tier, partition assignment, detector outputs, and configuration identifiers. For each released verified-positive replay row, evidence tables record payload family, sink-harness class, evidence tier, and harness outcome, allowing audit of the verified-positive label rule without tenant identities, callback domains, or exploitable sink details.

The package also releases the de-identification script, transformation manifest, validation checks, hash manifests, and reproduction scripts. Those scripts rebuild replay denominators with unresolved rows excluded, regenerate detector-overlap tables, and verify that published thresholds use benign calibration rows only. We omit tenant identities, reversible mapping keys, raw auxiliary logs, production weights, and exact private strings because they would re-identify organizations or expose production internals. The retained fields are the ones needed to reproduce accuracy, overlap, calibration, and label-evidence summaries from the released data.

Appendix C. Implementation and Edit Model

Frozen configuration. Each reported run pins the normalizer manifest Π , taxonomy/generator manifests, encoder checkpoint, cone axes, LSH probe seed, softmax temperature, effective count n_0 , ε smoothing floor, priors, mixture weights π , tenant calibration windows/thresholds, and edit-manifest version $\mathcal{E}_{\text{raw}}$. Released scripts recompute replay

metrics from de-identified rows and these configuration identifiers; raw tenant logs, reversible mappings, live-control weights, and exact private strings remain outside the artifact boundary.

Normalizer and edit model. Π is an ordered recorded-artifact normalizer, not a DNS-validity filter: it preserves the raw string, URL-like prefix/userinfo/port segmentation, Unicode/IDNA and case-normalization outcomes, percent/UTF-8 decode residues, and encoder token boundaries. $\mathcal{E}_{\text{raw}}$ is mined from observed normalization differences, verified payload variants, and public payload-corpus variants sharing the same executable core. Retained families cover scheme/prefix toggling, delimiters, percent/UTF-8 and hex/base encodings, case changes, Unicode confusables, quote/brace/comment fragments, and bounded DNS-label edits. Certificates are exact only for the fixed manifest version and deployed score path.

Calibration and stability check. For tenant t , CCD sets τ_α from benign calibration scores using the split-conformal order statistic for the target FPR and predicts positive only when $S_\pi(q(r)) > \tau_\alpha$. The calibrated-margin verifier bounds covered edit-ball movement by $\Delta_K(r) = n_0 A_\varepsilon L_Q B_K(r)$, where A_ε is the smoothed log-ratio envelope, L_Q bounds cone-sketch movement, and $B_K(r)$ bounds embedding rotation over K manifest edits. Unstable top- R support triggers exact all-cone scoring; SEC randomness only orders traversal and certificates require deterministic edit-ball closure.

Likelihood-ratio derivation. Let cone counts be $N=(N_1, \dots, N_M)$ with total count n_0 and empirical sketch $\hat{Q}(j)=N_j/n_0$. Under H_0 , $N \sim \text{Mult}(n_0, P_B)$. Under H_1 , a latent family $K \sim \pi$ is sampled and $N \mid K=k \sim \text{Mult}(n_0, P_{M,k})$; the Poissonized form gives the same ratio. With renormalized ε smoothing, all probabilities are positive.

Lemma C.1 (Score derivation for the cone-count statistic). *For each k , the component log-likelihood ratio equals $n_0[H(\hat{Q}, P_B) - H(\hat{Q}, P_{M,k})]$. Thus*

$$\log \frac{\sum_k \pi_k \mathcal{L}(N; P_{M,k})}{\mathcal{L}(N; P_B)} = S_\pi(\hat{Q}),$$

so thresholding S_π is the likelihood-ratio test for the benign cone-count model versus the learned positive mixture.

Proof sketch: Under $\text{Mult}(n_0, P)$, $\mathcal{L}(N; P) = [n_0! / \prod_j N_j!] \prod_j P(j)^{N_j}$. The multinomial coefficient cancels in the ratio $\Lambda_k = \prod_j [P_{M,k}(j)/P_B(j)]^{N_j}$. Taking logs and substituting $\hat{Q}(j) = N_j/n_0$ gives the cross-entropy form; independent Poisson counts cancel in the same way.

CAHO training and baselines. The deployed CAHO encoder fine-tunes all-MiniLM-L6-v2 (22.7M parameters; 384-d pooled output, ℓ_2 normalized) with a supervised contrastive orbit-collapse loss plus a binary auxiliary classification loss. Each batch samples two views per string under the replay manifest. Benign views come from $\mathcal{T}_0$ (case, delimiter, and token jitter).

TABLE 10. LLM BASELINE IDENTIFIERS USED IN THE MATCHED PRODUCTION REPLAY.

Family	Checkpoint identifier
70B instruct	meta-llama/Llama-3.3-70B-Instruct
32B instruct	Qwen/Qwen2.5-32B-Instruct
14B instruct	Qwen/Qwen2.5-14B-Instruct
8B instruct	meta-llama/Llama-3.1-8B-Instruct

Positive views come from leaf-specific $\mathcal{T}_1$ orbits (prefix/scheme rewrites, percent encodings, UTF-8 residues, hex/base forms, confusables, separator edits, comment/quote/brace fragments, and bounded DNS-label edits). We use AdamW with learning rate 10^{-4} , weight decay 10^{-2} , batch size 256, and up to 50 epochs; model selection uses validation-only performance at the target false-positive rate. After selection, the normalizer, encoder, 4096 cone axes, top-8 retrieval, priors, mixture weights, effective count, and thresholds are fixed before test and production replay. Held-out production rows are used only for evaluation. Baselines share the same recorded-string stream, normalizer closure, tenant/window calibration policy, partition protocol, and false-positive budgets; Table 10 lists the locally served LLM checkpoints.

Synthetic-validity checks. We compare synthetic and real positives by payload-family coverage, edit-family coverage, embedding-neighborhood overlap, nearest-neighbor leakage, and recall at the deployment threshold. Real-payload training provides the negative control: at a 10^{-4} false-positive rate it improves over LLM-synthetic training by 2.4 percentage points. The real replay therefore still contains additional signal, while the synthetic pipeline remains close enough to train a deployable detector without collecting raw tenant payloads.

Appendix D.

Ethics, Disclosure, and Open Science

De-identification. HIB-Real contains the complete retained evaluation replay after de-identification, not a sampled summary or synthetic surrogate. The released fields include the string, canonical form, coarse timestamp bucket, label-support tier, partition assignment, detector outputs, stable identifiers, benchmark scripts, and label-evidence tables with payload family, sink-harness class, evidence tier, and harness outcome. Sensitive spans are replaced by surrogates that preserve attack-relevant character classes, lengths, delimiters, casing, and encoding motifs without exposing organization-identifying values.

We release the de-identification script, transformation manifest, validation checks, and aggregate preservation report alongside the rows. The preservation record includes source/window counts, tenant-count aggregates, label counts, length buckets, character-class masks, delimiter/encoding motifs, canonicalization outcomes, score bins, and threshold decisions. These fields retain the executable-string structure and replay accounting while omitting organization-identifying joins, reversible mapping keys, and raw operational logs.

Risk handling. Positive examples contain payload strings that may execute in matching sinks and must be treated as untrusted input. Artifact guidance therefore calls for offline inspection, disabled DNS/network egress unless explicitly needed, no copy/paste into shells or query consoles, and render-only tools. CCD is an early enforcement and evidence signal at a configured alert budget; downstream command, SQL, template, URL-fetch, and alert-action sinks still need ordinary data/code separation.

Disclosure and production handling. The live deployment reported tenant-facing findings through the existing security-response workflow for the affected organizations. The paper does not publish tenant identities, unresolved strings, or exploitable private sink details. Public anchors are already public, fixed, or advisory-backed incidents used to test taxonomy coverage outside the deployment, not new disclosures. Controlled harnesses used for labeling ran with side effects blocked or restricted to researcher-controlled callback endpoints, and remediation decisions remained with service owners.

Label scope. The benchmark keeps labels, harness outcomes, and response effects separate. Released records identify benign, unresolved, and verified-positive strings; harness and response fields preserve the operational detail used in the main production-overlap analysis. Locally hosted LLMs serve as synthetic-variant generators, consistency triage, and offline baselines under the same partition and threshold protocol. Resolved labels and headline counts come from controlled sink evaluation, masked review, and the written rubric.

Appendix E. Reproducibility Path

The benchmark package preserves the accounting behind the reported results. Starting from the released de-identified rows, scripts rebuild denominators, partition membership, benign-calibrated τ_α , resolved recall and false-positive rates, detector outputs, and overlap views while keeping unresolved entries separate. This path recovers the reported 93.5% recall at a 10^{-4} false-positive rate over the 363,401 verified executable-semantics positives. Live/shadow evidence is released only as audited aggregate counts and masked summaries, not raw live streams. An independent replay of release d2a5b6 recomputed count metrics within 0.01%; hardware differences affected latency measurements but not classification metrics.

For the live study, the package preserves source counts, tenant summaries, configuration and runtime identifiers, input-parity checks, masked review summaries, harness identifiers, and privacy-preserving case-chain logs. These records support the matched-input comparisons, controlled outcomes, and case chains without exposing organization-identifying data. Public anchors (Cockpit remote-login RCE, enterprise git push RCE, Koa ctx.hostname, and legacy host/diagnostic reports) test taxonomy scope; they are not counted as HIB training or production positives.

TABLE 11. REPRODUCIBILITY BOUNDARY FOR THE MAIN CLAIMS.

Claim	Released or recomputable	Private by design
Replay accuracy	Complete de-identified HIB-Real replay rows used for replay accuracy, drift, calibration, and baseline metrics; de-identification script, labels, splits, thresholds, detector outputs, scripts, fixed-FPR curves	Tenant identities, reversible mappings, raw operational logs
Baselines	Model/config identifiers, fixed splits, calibration protocol, metric and timing outputs	Closed control internals, private weights or adapters, local serving traces beyond summaries
Live overlap	Overlap counts, overlap cells, masked case-chain summaries, harness outcome fields	Raw tenant logs, exact sensitive strings, response-owner records
Sink evidence	Sink family, evidence tier, controlled harness result	Exploitable sink details, callback domains, credentials, tenant names

Live-overlap accounting. Unresolved items remain a separate state rather than silently disappearing from the evaluation. Main replay metrics use resolved rows only; unresolved rows remain available for prevalence, drift, and stress tests. Incumbent controls see known request-path tokens, while CCD scores retained strings before shell, template, alert, or query interpretation. The measured boundary is the moment when a retained host string is still data but is about to enter code-like reuse; CCD supplies a low-latency control for surfacing execution-capable strings missed by signatures at the incumbent review volume.

Appendix F. Additional Scope and Mimicry Checks

Synthetic-real validity. Synthetic positives train the model; HIB-real rows determine prevalence and impact. Across 47 structural features, 43 have no significant synthetic-vs-real difference at $p > 0.05$; in CAHO space, FID is 0.23 versus a real-real baseline of 0.19, MMD is 0.0041, and permutation tests do not reject equality. Adjusting three generator budgets that differ from HIB-real (length, TLD diversity, Base64-like frequency) moves recall from 93.5% to 94.7%, reducing the real-training gap from 2.4 pp to 1.2 pp; the deployed model uses the untuned base generator. Exact public-text overlap is 127/148,392 strings (0.086%), paraphrases are detected within 1.6 pp of originals ($p=0.24$), and LLM-specific n -gram excess over a PCFG generator is 0.3%.

Adjacent-field scope. Performance follows field structure. At 10^{-4} FPR, CCD is strongest where inputs are bounded, structured, and obfuscation-heavy: hostnames (93.5% vs. 90.8% for CSI), email headers (84.1% vs. 79.4% for Spam-Net), and full URLs (90.3% vs. 89.2% for urlBERT). It is not intrinsically better for unbounded log text, where LogBERT slightly leads (84.7% vs. 84.5%); we treat this as a boundary check, not a general-security-log claim.

Table 12 summarizes the hostile-mimicry stress test; the attacker model and interpretation follow below.

TABLE 12. WHITE-BOX HOSTILE-MIMICRY RECALL.

Suite	CCD recall@ 10^{-4}	CCD recall@ 10^{-5}	Best baseline@ 10^{-4}
Marker-preserving mutation	99.2%	97.5%	85.4%
Marker-erasing equivalent	94.1%	89.3%	62.1%
Cone-escape optimization	91.5%	84.2%	45.8%
DNS-label segmentation	96.8%	92.1%	58.3%
Punycode/IDNA attacks	98.5%	95.4%	71.2%

Hostile mimicry setup. The mimicry suites search for r' with $d_{\mathcal{E}_{\text{raw}}}(r, r') \leq K$ that lowers $S_{\pi}(r')$ or flips $h(r')$ while preserving payload semantics. Heuristic mimicry uses delimiter, token, suffix, encoding, Unicode, and prefix edits; black-box mimicry uses fixed-budget score-only or label-only local search; generator mimicry slots encoded payload fragments into benign-looking hostname skeletons. All suites set thresholds on held-out benign data at the same FPR, so crossing baseline rankings measure camouflage robustness rather than clean replay recall alone.

Hostile mimicry results. Table 12 shows that across marker-preserving changes, marker-erasing equivalents, cone escape, DNS-label segmentation, and Punycode/IDNA, CCD’s weakest cell is 84.2% at a 10^{-5} false-positive rate. The strongest baseline reaches 45.8% at 10^{-4} , matching the cone-sketch ablation in Table 6. CAHO’s geometry keeps mimicry outside benign cones.